MODULE 06 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Pods

The smallest thing Kubernetes runs: what it shares, what it contains, and how it lives and dies.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 6 of 35

admin@sthithapragna.com

A Pod is **a shared context** for one or more containers: one IP address, shared volumes, one lifecycle.

Why Pods, and not bare containers?

- **The smallest deployable unit.** Kubernetes creates, schedules and deletes Pods, never single containers.
- **Co-location.** Every container in a Pod lands on the same node, together.
- **Usually one app container.** Extra containers are helpers for that one app.
- **Scale by adding Pods,** not by adding containers to a Pod.

> ON THE EXAM
>
> Hierarchy to remember: cluster, then nodes, then Pods, then containers.

What containers in one Pod share

THING	SHARED?
Network namespace: IP address, ports, localhost	Yes, always
IPC namespace: shared memory	Yes
Hostname (UTS)	Yes
Volumes	Yes, if each container mounts them
Process list (PID namespace)	Only with shareProcessNamespace: true
Root filesystem	No. Each container has its own image
CPU and memory limits	No. Set per container

ON THE EXAM

Network and storage are the two things the exam says containers in a Pod share.

Consequences of sharing a network

- **One port space.** Two containers in a Pod cannot both listen on port 8080.
- **localhost works.** A helper reaches the app at localhost, no Service needed.
- **One Pod, one IP.** Other Pods reach it by that IP, or through a Service.
- The shared namespaces are held open by a tiny **pause** container, created by the runtime.

> ON THE EXAM
>
> Different Pods never clash on ports: each Pod has its own IP address.

Two containers in one Pod, or two Pods?

SAME POD

Tightly coupled

- Must run on the same node
- Share files or localhost
- Start and stop together
- Example: app plus its log shipper

SEPARATE PODS

Loosely coupled

- Scale independently
- Fail independently
- Talk over a Service
- Example: web front end and database

THE TIEBREAKER

One Pod only when they must share a node and a lifecycle. Otherwise, separate Pods.

Four kinds of container in a Pod

KIND	WHERE DECLARED	BEHAVIOUR
App	containers	The main program.
Init	initContainers	Runs to completion, in order, before the app.
Sidecar	initContainers, restartPolicy: Always	Starts first, keeps running, stops last.
Ephemeral	added to a running Pod	Temporary, for debugging. Never restarted.

ON THE EXAM

Native sidecars: stable since Kubernetes 1.33. Init containers that never finish.

Quayside's booking API Pod, all three at once

```
spec:
  initContainers:
  - name: migrate
    image: quayside/booking-migrate:2.4.1
  - name: log-shipper
    image: fluent/fluent-bit:4.0
    restartPolicy: Always
  containers:
  - name: api
    image: quayside/booking-api:2.4.1
```

3 A regular init container: migrates the database schema, then exits.

7 restartPolicy: Always turns this init container into a sidecar.

9 The app starts only after migrate succeeds and the sidecar has started.

Start-up and shut-down order

01
Init containers
Run one by one; each must succeed.

›

02
Sidecars start
In order, and keep running.

›

03
App containers
Start once all of the above are done.

›

04
Shut-down
App containers stop first.

›

05
Sidecars stop
Last, in reverse order.

ON THE EXAM

Sidecars wrap the app: first in, last out. So logs and proxies cover the app's whole life.

Old-style sidecar versus native sidecar

OLD STYLE

A second app container

- Listed under containers
- No start-up ordering
- May stop before the app finishes
- Keeps Job Pods from completing

NATIVE, STABLE SINCE 1.33

An init container that keeps running

- initContainers, restartPolicy: Always
- Guaranteed to start first
- Stops after the app
- Job completes when the app exits

THE TIEBREAKER

If a Job never finishes because a helper keeps running, the fix is a native sidecar.

Multi-container patterns

PATTERN	THE HELPER...	EXAMPLE
Sidecar	extends or enhances the app	Log shipper, service mesh proxy
Ambassador	proxies the app's calls to the outside world	Local proxy to a remote database
Adapter	reshapes the app's output for others	Converts custom metrics to Prometheus format
Init	prepares things before the app starts	Schema migration, fetching config

ON THE EXAM

All three helper patterns are just containers in the same Pod. Only the job differs.

A Pod is born, runs and dies. It is **never resurrected**. A replacement is always a new Pod.

The five Pod phases

PHASE	MEANS
Pending	Accepted, but containers are not running yet: waiting to schedule, or pulling images.
Running	Bound to a node; at least one container is running, starting or restarting.
Succeeded	All containers exited successfully and will not be restarted.
Failed	All containers have stopped, and at least one failed.
Unknown	The Pod's state cannot be obtained, usually lost contact with the node.

ON THE EXAM

CrashLoopBackOff, ContainerCreating and Terminating are not phases. They are what the STATUS column shows.

Container states and their reasons

STATE	COMMON REASONS
Waiting	ContainerCreating, ImagePullBackOff, ErrImagePull, CrashLoopBackOff
Running	The process is running. Probes may still fail.
Terminated	Completed, Error, OOMKilled, with an exit code

> ON THE EXAM
>
> kubectl get pods shows the most important container reason in STATUS, which is why it looks like a phase.

restartPolicy: set once for the whole Pod

VALUE	RESTARTS CONTAINERS...	TYPICAL USE
`Always (default)`	whenever they exit, even successfully	Deployments, StatefulSets, DaemonSets
`OnFailure`	only after a non-zero exit	Jobs that should retry in place
`Never`	never	Jobs that retry by making new Pods, or one-off tasks

ON THE EXAM

Restarts back off: 10, 20, 40 seconds and so on, capped at five minutes, reset after ten healthy minutes.

What CrashLoopBackOff really means

- **A waiting reason, not a phase.** The kubelet is pausing before the next restart.
- **The container keeps exiting.** It started, then crashed or was killed, again and again.
- **Usual causes:** the app crashes at start, bad config, a missing dependency, OOMKilled, a failing liveness probe.
- **Diagnose with** kubectl logs --previous and kubectl describe pod.

> ON THE EXAM
>
> CrashLoopBackOff says the container starts and then dies. ImagePullBackOff says it never started at all.

Graceful termination

01
Delete
Pod marked Terminating; removed from Service endpoints.

02
preStop hook
Runs first, if defined.

03
SIGTERM
Sent to each container's main process.

04
Grace period
30 seconds by default, to finish work.

05
SIGKILL
Anything still running is killed.

ON THE EXAM

Apps should handle SIGTERM and finish in-flight work. Otherwise every deploy drops requests.

> Quayside adds a caching proxy container to the booking API Pod. Both the API and the proxy are configured to listen on port 8080.
>
> The API starts normally. The proxy crashes repeatedly with the error: address already in use.

Why does the proxy fail?

A Another Pod on the same node is already using port 8080

B The Service can only route to one port per Pod

C kube-proxy reserves port 8080 on every node

D Containers in a Pod share one network namespace

◆ PAUSE AND DECIDE

Why does the proxy fail?

A Another Pod on the same node is already using port 8080

Each Pod has its own IP and port space. Other Pods cannot clash.

B The Service can only route to one port per Pod

Services can expose several ports, and a Service never blocks binding.

C kube-proxy reserves port 8080 on every node

kube-proxy reserves no such port inside Pods.

D Containers in a Pod share one network namespace

One IP, one port space: the second listener on 8080 collides.

THE SIGNAL

A port clash inside one Pod means shared network namespace. Change one container's port.

> Quayside's nightly manifest export runs as a Job. The team added a log-shipper as a second entry under containers.
>
> Since then, the export finishes its work each night, but the Job never completes. The Pod stays Running.

What is the best fix?

A Move the log shipper to initContainers with restartPolicy: Always

B Set the Pod's restartPolicy to Never so nothing restarts

C Add activeDeadlineSeconds so the Job is stopped after an hour

D Move the log shipper to initContainers, leaving restartPolicy unset

◆ PAUSE AND DECIDE

What is the best fix?

A Move the log shipper to initContainers with restartPolicy: Always

A native sidecar. The Job completes when the export container exits.

B Set the Pod's restartPolicy to Never so nothing restarts

The shipper still never exits, so the Pod still never finishes.

C Add activeDeadlineSeconds so the Job is stopped after an hour

Stops it, but the Job is then marked Failed every night.

D Move the log shipper to initContainers, leaving restartPolicy unset

A regular init container must finish first; the export would never start.

THE SIGNAL

A helper that never exits, blocking a Job: make it a native sidecar.

> kubectl get pods shows the payment worker with STATUS CrashLoopBackOff and RESTARTS 7. Its restartPolicy is the default.
>
> A colleague says the Pod is in the CrashLoopBackOff phase. You check the Pod's actual phase field.

What phase will the Pod most likely report?

A Pending, because the container is not currently up

B Failed, because the container keeps crashing

C Running, while the kubelet keeps restarting it

D Unknown, because its state cannot be obtained

◆ PAUSE AND DECIDE

What phase will the Pod most likely report?

A Pending, because the container is not currently up

Pending ends once the Pod is bound and its containers created.

B Failed, because the container keeps crashing

With restartPolicy Always, a Pod never reaches Failed this way.

C Running, while the kubelet keeps restarting it

Running includes containers being restarted; the reason is CrashLoopBackOff.

D Unknown, because its state cannot be obtained

The kubelet is reporting fine. Unknown means lost contact.

THE SIGNAL

Phase and STATUS are different fields. Restarting in place keeps the phase at Running.

> Quayside is writing a Job for a monthly fare recalculation. If the container exits with an error, it should be restarted in the same Pod.
>
> If it exits successfully, it must not run again.

Which restartPolicy should the Job's Pod template use?

A Always, so any exit triggers a restart

B OnFailure, so only error exits restart

C Never, so each failure creates a new Pod

D Leave it unset and accept the default

◆ PAUSE AND DECIDE

Which restartPolicy should the Job's Pod template use?

A Always, so any exit triggers a restart

Would rerun after success, and a Job does not accept it.

B OnFailure, so only error exits restart

Restarts in place on error; leaves a successful exit alone.

C Never, so each failure creates a new Pod

Retries happen, but in new Pods, not the same one.

D Leave it unset and accept the default

The default is Always, which a Job rejects.

THE SIGNAL

'Restart in the same Pod, only on error' is OnFailure, word for word.

Module 6 in five lines

- A Pod is **one IP and shared volumes** for containers that must live together.
- **Init** runs first and finishes; a **native sidecar** is an init container with restartPolicy: Always.
- Five phases: **Pending, Running, Succeeded, Failed, Unknown**. CrashLoopBackOff is not one.
- **restartPolicy**: Always by default, OnFailure or Never for Jobs.
- Delete sends **SIGTERM**, waits **30 seconds**, then SIGKILL.

You rarely create Pods yourself. Next: **ReplicaSets and Deployments**, which create them for you.